深圳大学考试答题纸

(以论文、报告等形式考核专用)

二〇二四～二〇二五　学年度第　二　学期

课程编号	1502880001	课程名称	基于 Web 的编程	主讲教师	Basker George	评分	

学号	2022155033	姓名	洪子敬	专业年级	2022 级软件工程（腾班）
学号	2022150102	姓名	温奇炜	专业年级	2022 级计算机科学与技术
学号	2022150120	姓名	麦子垣	专业年级	2022 级计算机科学与技术

教师评语：The mark obtained for different categories as per the grading policy are as follows

序号	**评分点　（分值）**	**成绩**
1	功能实现完整性：页面结构、模块功能是否齐全且运行良好（40 分）	
2	用户体验与交互设计：页面交互流程是否清晰合理、易用（10 分）	
3	视觉美感与细节：页面设计统一、美观、有明确风格（5 分）	
4	性能与适配设计：兼容多浏览器、适配不同屏幕的能力（5 分）	
5	团队协作与贡献：分工清晰、每人独立完成指定模块（5 分）	
6	功能创新与亮点：新颖设计、互动体验、个性功能（5 分）	
7	整体分析与平台设计：项目目标、用户流程、技术选型等（5 分）	
8	个人模块设计与实现：具体 HTML、CSS、JS 设计分析与实现细节（20 分）	
9	项目总结与反思：项目协作过程体会、改进建议等（5 分）	
合计	100	

Basker George
30 June 2025

题目：Campus Social Platform

组员个人总分权重分配表：

排序	姓名	学号	项目个人权重	个人最后得分	评分人
1	洪子敬	2022155033	100%		
2	温奇炜	2022150102	95%		
3	麦子垣	2022150120	75%		
4					
5					

任务分工及所完成的状况。

（2 人总分为 190% *教师评分, 3 人总分为 270% *教师评分）

个人总分权重分配表

（1 人总分为 100%，2 人为 200%, 3 人为 270%，4 人为 350%，5 人为 430%）

排序	姓名	学号	项目个人权重
1	洪子敬	2022155033	100%
2	温奇炜	2022150102	95%
3	麦子垣	2022150120	75%
4			
5			

我组成员总共__3__名，权重总和为：270 %

Database Major Project Report

一、 Experimental Overview

For this major project, we focused on developing CampusConnect, a campus social networking webpage designed to integrate multiple functionalities: user systems for regular users, dynamic content publishing and browsing, interactive social features, and a clock-in mechanism. By leveraging front-end technologies including HTML, CSS (Tailwind CSS), and JavaScript, and integrating with backend APIs for data interaction with a local database, we successfully completed the development of this campus social platform as initially envisioned. The implementation has effectively realized our goal of creating a comprehensive, interactive space for campus social activities.

二、 Experimental Environment

2.1 Development tools

- Text editor：Visual Studio Code
- Browser：Google Chrome

2.2 Technology stack

- Frontend：HTML、CSS（Tailwind CSS）、JavaScript
- Backend：Express、mysql2/promise、bcryptjs、Multer、cors、dotenv

三、Web Design

The overall CampusConnect web page design can be roughly divided into five parts, namely the general visitor section, the user system section, the dynamic posting and browsing section, the interaction and social section, and the personalized section. The following is a detailed introduction to these sections.

3.1 Ordinary User Section

Overall implementation: To ensure a good experience for tourists, compared with the page after logging in, the browsing of posts, the browsing of comments, and the browsing of the personal main page are retained here. Other functions, including posting, following, favoriting, private messaging, liking, etc., are all unavailable. When a non - logged - in user clicks, a warning "Please log in/register first to use this function" will pop up, and the login and registration page will be displayed for the user to log in or register.

Specific effects: When in the unlogged-in state, the overall interface is as shown in Figure 1:

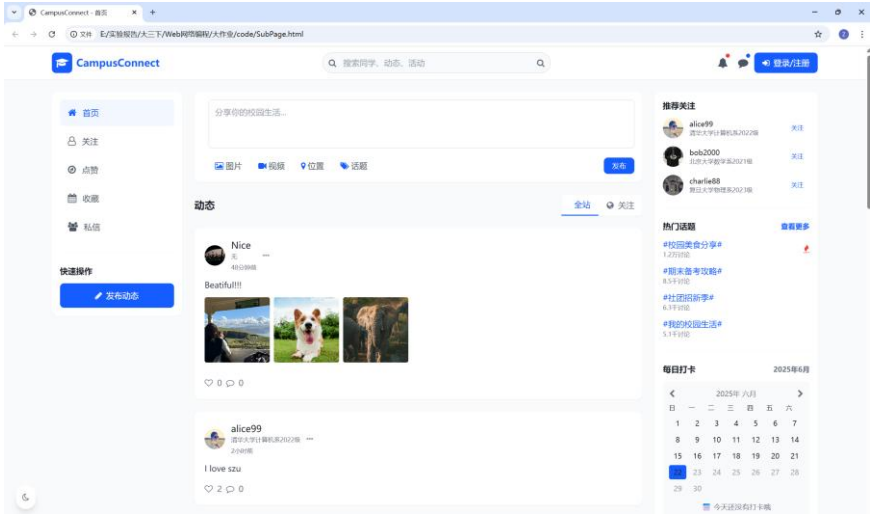


Figure 1. Unlogged-in web page

3.2 User System Section

3.2.1 Registration section

Overall implementation: Unregistered users can register by clicking the "Login/Register" button in the upper right corner of Figure 1. The registration content includes "Upload user avatar", "Nickname", "Email", "Personal profile", "Interest tags", "Password", "Confirm password", and service terms. Users upload their avatars in the form (otherwise, the default avatar will be used) and fill in their personal information (personal nickname, email, and password are required, and others are optional). Specifically, JavaScript is used to bind the form, and the results are passed to the backend API through /api/register to add the new user information to the database.

Specific effect: After the code is written, the effect of clicking register and filling in information is as shown in Figure 2:

Figure 2. Registration Page - Initial Page (left), Filling in Information (right)

Note: Since web browsers come with built - in password show and hide buttons, to avoid duplication, the password hiding and showing function is directly removed here, and the browser's built - in function is used instead.

The password display in HTML uses password strength evaluation of different levels and is evaluated and updated in real - time through JavaScript listening. The password strength calculation logic function is as follows:

```
// Calculate password strength
function calculatePasswordStrength(password) {
    let strength = 0;
    if (password.length >= 8) strength++;
    if (/[a-z]/.test(password)) strength++;
    if (/[A-Z]/.test(password)) strength++;
    if (/[0-9]/.test(password)) strength++;
```

```
    if (/[^a-zA-Z0-9]/.test(password)) strength++;
    return strength;
}
```

3.2.2 Login section

Overall implementation: Similar to the registration function, users can log in by filling in their email and password in the login box and verifying them through the backend interface. However, after obtaining the FormData from the form fields in the login section, the current user information needs to be stored in the local localStorage service (although the storage space is small, it's sufficient). At the same time, update the state of the global user variable to facilitate the use of other interfaces that need to obtain the current user information. By using localStorage and the global user variable, the login state can be retained even after refreshing the web page. Specifically, check the state after the document is loaded. If there is corresponding user information in localStorage, update the global state.

At the same time, after successful login, as shown in Figure 1, the web page has not been logged in successfully at this time, so relevant user information is not displayed. Therefore, corresponding JavaScript code needs to be added to monitor the user state. When the user logs in successfully, update the user information, including replacing the "Login/Register" button with the current user's information, adding the current user's information bar on the right, and adding the user avatar when commenting and posting dynamic content.

Specific effect: After completing the writing of the login code, clicking "Login" and filling in the information will have the effect as shown in Figure 3:

Figure 3. Login Page. Initial Page (left), Fill in Information (right)

After a successful login, after completing the real-time update of all the dynamic elements mentioned above, the effect shown in Figure 4 can be obtained:

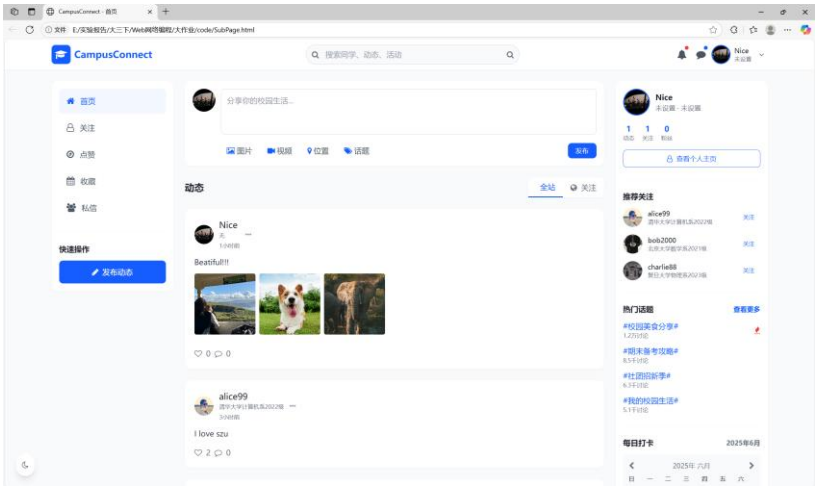


Figure 4. The web page after logging in

3.2.3 Administrator section

Overall implementation: As mentioned before, we use global variables to store the information of the current user. Here, the "type" field in the global variable is used to represent the identity of the current user. 0 represents an ordinary user, and 1 represents an administrator (the same applies to the user table). When the user is an administrator, this system does not grant the administrator the permission to ban users, but only gives the function of deleting comments. As shown in Figure 5, click the menu option on the right side of the dynamic post. Generally, if the user is not the owner of the post, the delete button will not be displayed for deletion. However, as long as it is detected that the current user is an administrator, the delete function will be available after opening the menu bar.

Specific effects: Log in with accounts of different permissions respectively, and you can see different menu displays as shown in Figure 5:

Figure 5. Administrator user effect (left), ordinary user (right)

3.2.4 Personal omepage function (including editing personal information and posts)

Overall implementation: For overall aesthetics, an independent personal homepage HTML is added based on the main HTML. As mentioned before, each post stores the user ID of the current post. Therefore, when making a long - link jump through the a tag, the corresponding user information can be found from the backend according to this user ID, and the corresponding HTML content can be supplemented through JavaScript, thus rendering a user page. In addition, an editing function needs to be provided to the user. When it is determined that the current user is the same as the user of the post, a "Edit Profile" button will be displayed, providing the current user with an opportunity to edit personal information. Of course, besides that, the currently logged - in user can also enter through the human - shaped icon in the drop - down menu in the upper - right corner or the "View Personal Homepage" button on the right sidebar. As shown in Figure 6, the current user has multiple ways to enter the personal homepage.

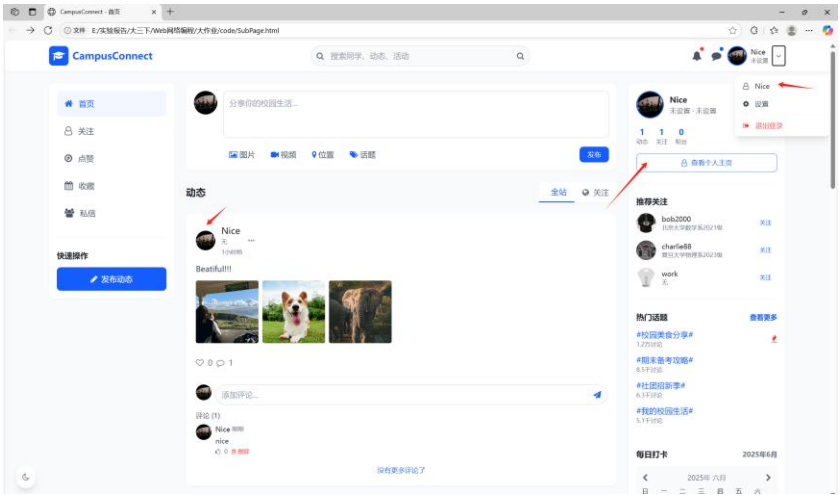


Figure 6. Various ways to enter the personal homepage

Specific effects: When the currently logged-in user clicks on the avatar of another user, they will enter the personal homepage of that other user. The effect at this time is as shown in Figure 7. If the currently logged-in user enters their personal homepage through various methods in Figure 6, as shown in Figure 8, an additional "Edit Profile" button will be displayed in the middle. Clicking this button will pop up a modal box for editing the profile, and modifying the personal content above can directly modify the database.

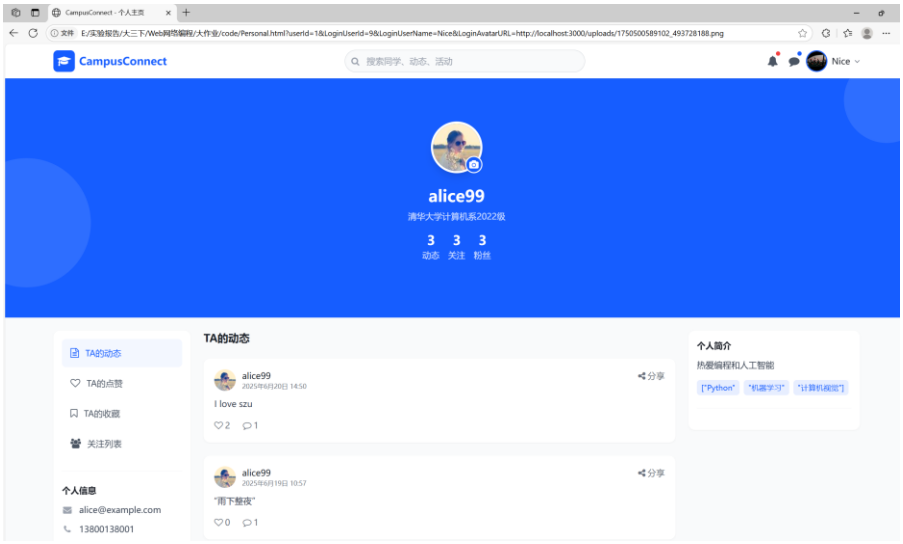


Figure 7. Personal Homepage of Non-Logged-in Users

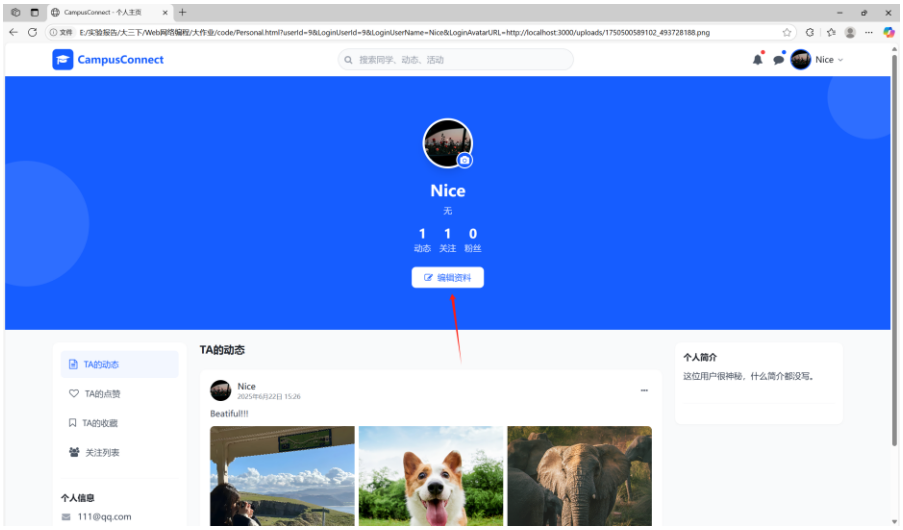


Figure 8. The personal homepage of the logged-in user

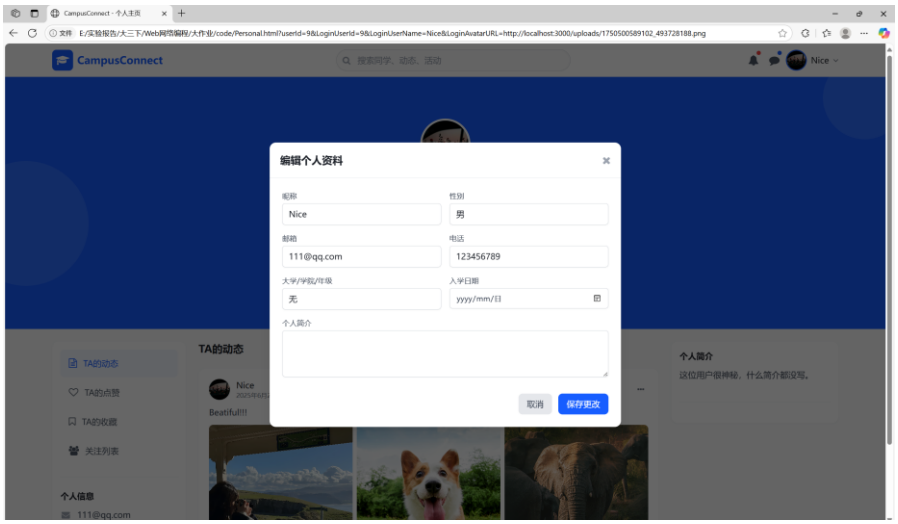


Figure 9. Edit Profile Modal

Of course, it is not difficult to find through the comparison of Figure 7 and Figure 8 that when the current user is a logged-in user, the share button in the upper right corner of the post changes to a menu with three dots. By clicking this menu, functions such as post editing (text modification and new picture upload) and deletion can be achieved. Specifically, it is also implemented by binding to the backend API. The specific effect is shown in Figure 10:

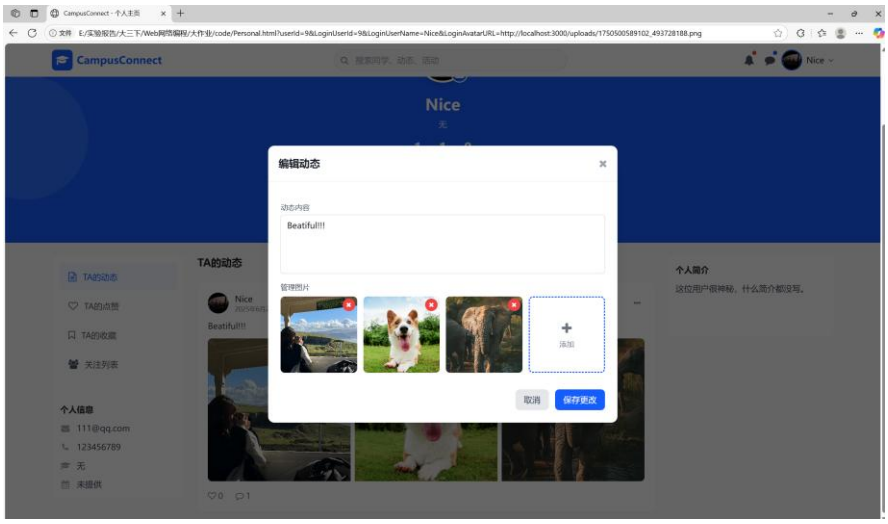


Figure 10. Edit Dynamic Modal Box

3.3 Dynamic Publishing and Browsing Module

3.3.1 Dynamic publishing section

Overall implementation: As shown in Figure 1, there are two places for dynamic posting, namely the quick operation on the left and the posting module box above the middle. To simplify the operation, a unified operation is carried out here. Clicking on either place will pop up a modal box. Data is packaged by binding the text box and picture upload box on the modal box. However, it should be noted that the picture data needs to be transformed. Specifically, by writing a backend interface, the current picture is stored in a specific path. At the same time, the current server path + relative path is used as the picture data, which facilitates unified operation of the picture data. After the post is successfully posted, that is, after the post is successfully stored in the database, the dynamic stream in the center of the homepage will be re-rendered, making it more real-time.

Specific effects: As shown in Figure 11, you can fill in the text information of the post and upload one or more pictures. The location and topic have not been implemented due to time constraints. Click "Publish" to post the dynamic.

Figure 11. Dynamic Publishing Modal - Initial Page (left), Filling in Information (right)

3.3.2 Dynamic like function

Overall implementation: Add a corresponding likes table to the database, and add a like_count

attribute to each dynamic post to display the real-time like effect (the number of likes for the current post). At the same time, a corresponding click event needs to be bound to it. When the currently logged-in user likes the post, call the backend interface /api/like to update the likes table and the post table, and replace the current hollow icon with a solid icon to indicate a successful like**.** When unliking, call the backend /api/unlike to update the like table and post table, and replace the current solid icon with a hollow icon to represent the successful unliking.

Specific effects: After the like function is implemented, the effects before and after liking are shown in Figure 12:

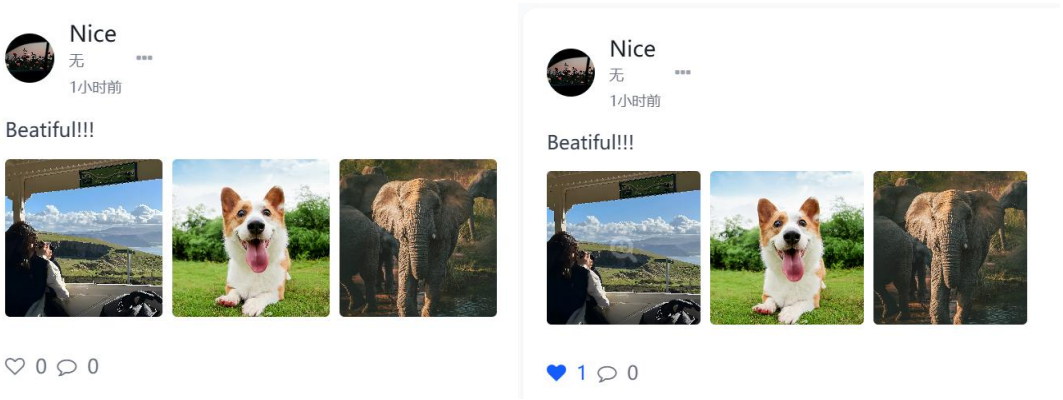


Figure 12. Before Liking (left) and After Liking (right)

3.3.3 Comment function (Post comments + Like comments)

Overall implementation: To implement the comment function, a modal box is added here, and the click event of the icon is bound. When the user clicks the comment area icon while hanging up, the comments corresponding to the current post will be loaded from the backend /api/comments (by storing the post ID when rendering the post, so that the comments of which post can be correctly identified). At this time, not only the comments need to be rendered, but also the avatar and username of the current user need to be rendered to represent which user posted the comment. In addition, a comment like function is added. This function is similar to the previous dynamic like function, but here a like_count attribute is added to the comment, and an additional comment like table is needed to store the relationship table between the user and the comment.

Specific implementation: After completing the previous code writing, click the comment icon and post a comment. The effect is shown in Figure 13 and Figure 14:

Figure 13. Effect of clicking the comment section icon

Figure 14. Effect of Posting a Comment

Similarly, liking a comment can achieve the effect shown in Figure 15. To avoid confusion with liking a post, the icon has been changed from the original heart to a thumb here.

Figure 15. Effect of Comment Likes

3.3.4 Delete function

Overall implementation: Since it can be published, a deletion function should also be added here. For posts, as mentioned before, when clicking the menu next to the post, if the current permissions are met (personal/ administrator), a "Delete" button will be displayed for the user to click to delete. At this time, deleting the post will delete the corresponding comments and comment likes, and update the relevant information of the corresponding user in the database. For the deletion of comments, different from posts, after deletion, not only the user table needs to be updated, but also the information in the post table needs to be updated**.**

Specific effects: The delete buttons for posts and comments after adding the delete function are shown in Figure 16:

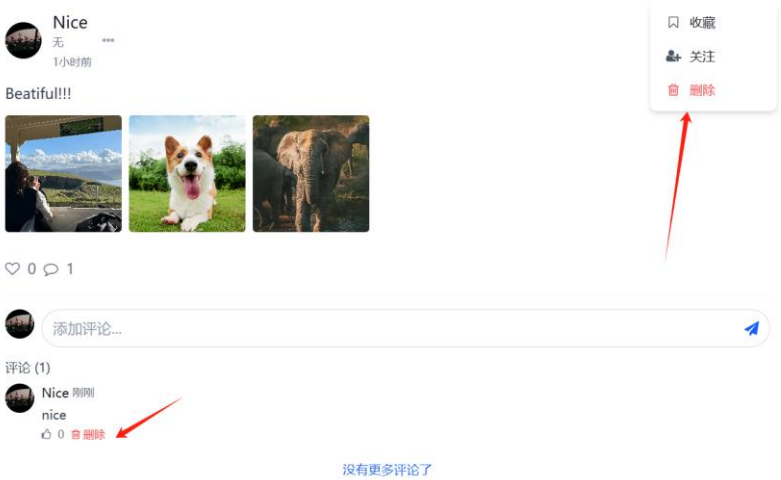


Figure 16. Effect of the delete icon

3.4 Interactive and Social Module

3.4.1 Follow function

Overall implementation: To better promote user communication, a follow feature is provided for users. Users can follow the user of the current post by clicking the "Follow" button on each post. After following, they can also unfollow the current user by clicking the "Unfollow" button at the same position in the menu again. By adding a follower count and a following count to each user table, and creating a new follow table to associate each pair of related users, the two quantities mentioned above can be counted through this table. The follower count and following count will be displayed on the right sidebar after the user logs in successfully. At the same time, these two quantities will also be displayed on each user's personal homepage.

Specific effects: As shown in Figure 17, click the menu button of the current post, and then click "Follow". At this time, the icon will change from the original one to a green icon without a plus sign, indicating that you have followed. Of course, the number of followers and following are not set to be displayed in real - time after following. They will only be updated when you log in next time, but this will not affect the actual effect.

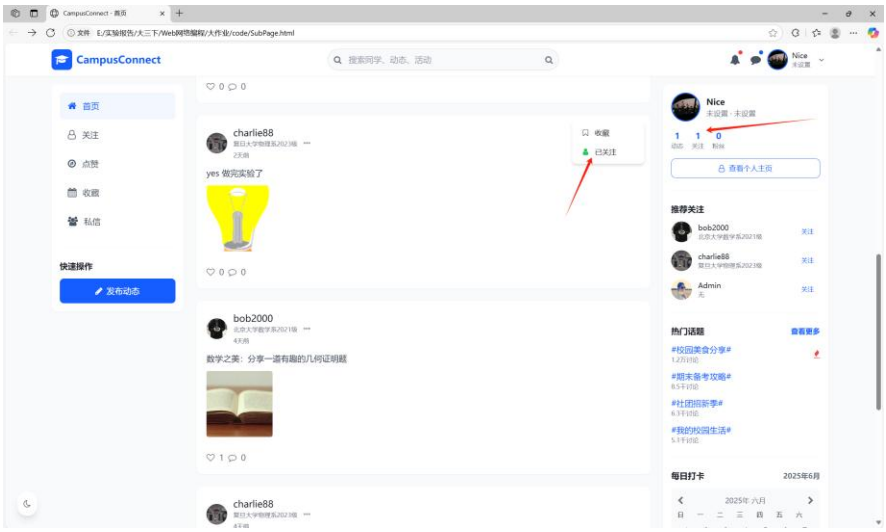


Figure 17. Follow effect

At the same time, for the convenience of the current user to view their followed objects, a function button "Follow" is added on the left. Clicking this button will query in the background and return the results to be displayed in the modal box. Users can unfollow the user in the modal box. The effect is shown in Figure 18.

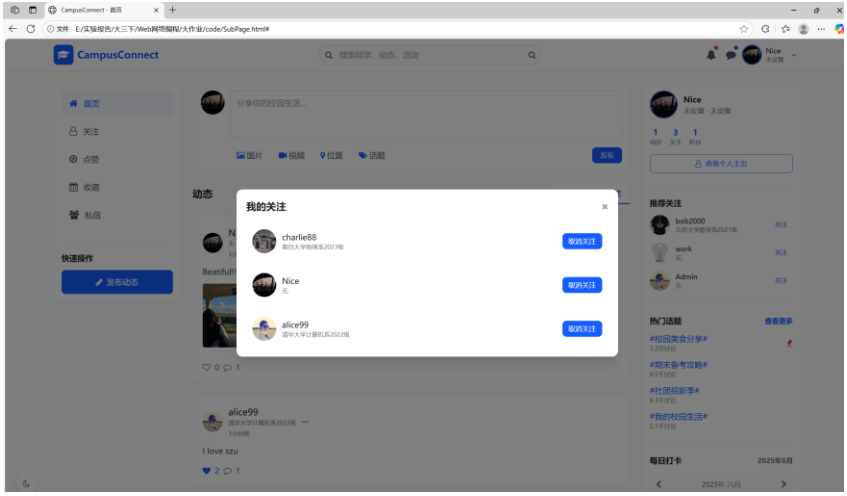


Figure 18. Current user's follow list

3.4.2 Switch between home page dynamics and whole site dynamics

Overall implementation: Add a toggle button in the middle of the post stream. By clicking this button, users can switch between "全站动态 (全站动态 means all-site dynamics)" and "关注动态 (关注动态 means followed dynamics)". The actual implementation is quite simple. Based on the original setup, add one more listener. If either of the two options is switched, re-render the post stream. However, there are differences between the two. An additional interface needs to be written for "关注动态 (关注动态 means followed dynamics)" to only filter and render the posts of the current user's followed objects. Other parts are basically the same.

Specific effects: By default, the website enters from "全站动态". When "关注" is clicked, the post rendering will be switched, as shown in Figure 19：

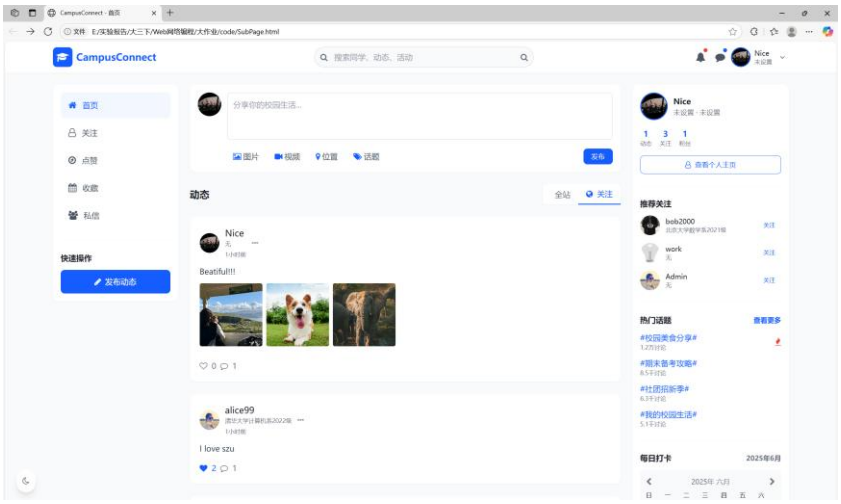


Figure 19. Focus on Dynamic Switching

3.4.3 Implementation of private message function

Overall implementation: Since only the interface needs to be implemented in this section, the submitted conversation records are directly rendered here. However, to add some interactive elements, users can switch back and forth between multiple dialog boxes. The rest are all static, and the implementation is relatively simple. It can be achieved by binding events to a modal box. In addition, on the main interface, by binding the "Private Message" button on the left sidebar, clicking the button will trigger the corresponding event, thus displaying the private message dialog box.

Specific effect: Click the "Private Message" button on the left sidebar, and the display effect is as shown in Figure 20:

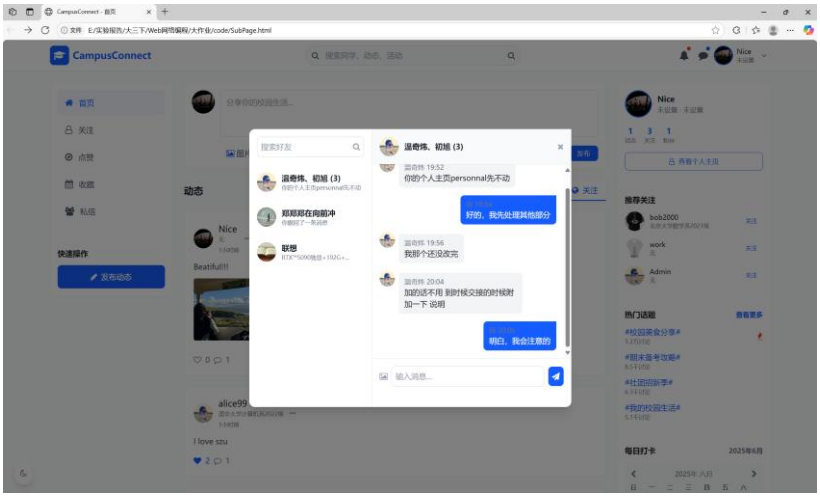


Figure 20. Private message effect

3.4.4 Theme Switch (Night Mode)

Overall implementation: The theme setting is quite simple. By setting the original form bar, title bar, navigation bar, etc. to another color system, the effect of different themes can be achieved. Here, the color system is simply switched to black and gray (originally white and gray), with no other changes. There is only a moon - shaped button bound in the lower left corner. The color switching is achieved by binding events.

Specific effect: Click the "Switch to Night Mode" button in the lower left corner to achieve the effect shown in Figure 21:

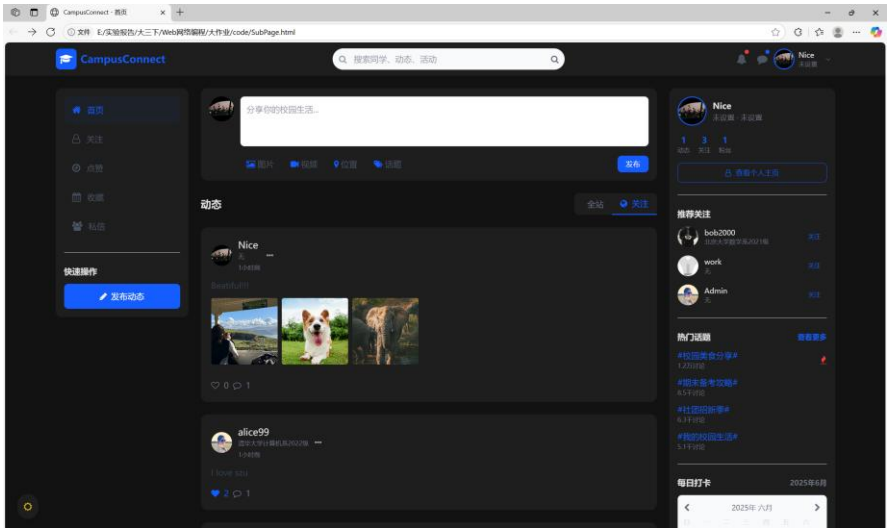


Figure 21. Night mode effect

3.4.5 Collection function

Overall implementation: Similar to the previous like logic, a function button for this will be added to the menu of each post. At the same time, an additional favorites table needs to be created in the database. By clicking the favorite button, the corresponding post content will be associated with the currently favoriting user. Initially, the star is unfilled. Similar to the previous situation, when the user clicks to favorite, the star will be filled with green, which is in line with daily usage.

Specific effect: Click the favorite button of the post, and the effect is as shown in Figure 22:

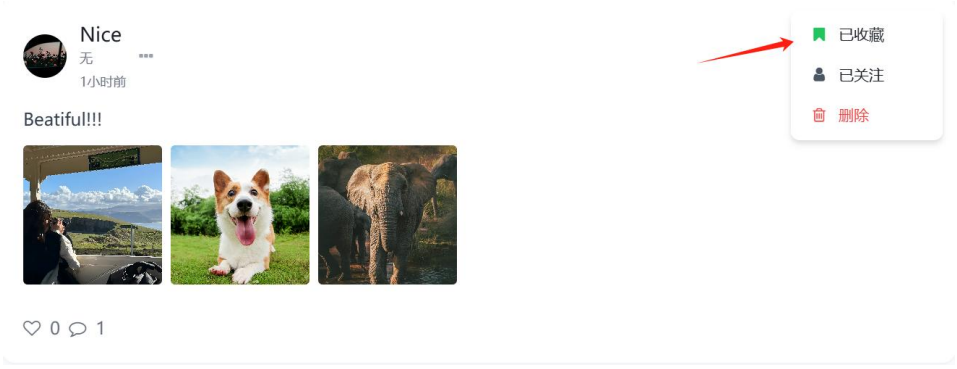


Figure 22. Effect of Post Collection

In addition, similar to the previous following function, in order to facilitate users to view the content they have collected, a "Collection" button is also set on the left sidebar. After the user clicks it, the collection list of the current user can be displayed. At the same time, the corresponding event response is also bound. The user can directly click "Uncollect" to cancel the collection of this post. The specific effect is shown in Figure 23.

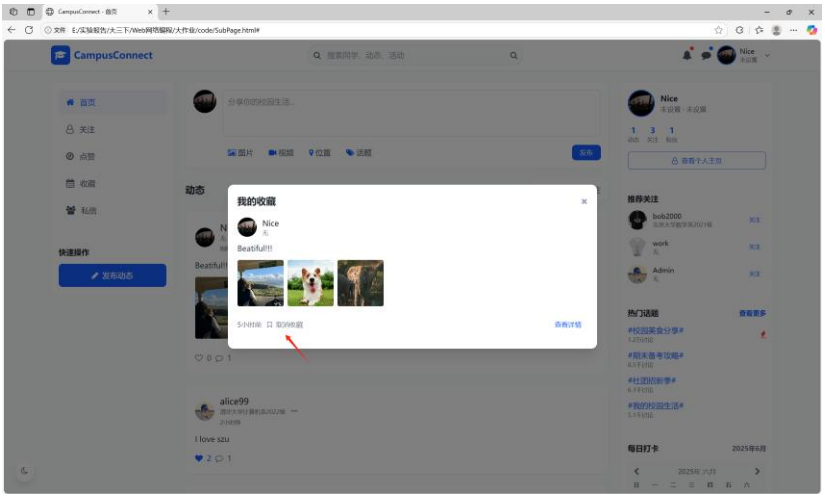


Figure 23. Posts favorited by the current user

3.5 Personalized section

3.5.1 User recommendation

Overall implementation: In order to promote communication among users, a user recommendation section is introduced here. However, the recommendation algorithm here is relatively simple. It simply recommends some users with a high number of posts and a large number of followers, randomly selects three from these users, obtains their user information, and returns it to the front - end for display. The core code is as follows:

```
const query = `
                SELECT
                    u.user_id, u.username, u.avatar_url, u.school_info,
                    u.follower_count, u.post_count,
                    0 as is_following
                FROM users u
                WHERE u.user_id NOT IN (SELECT following_user_id FROM Follows
                 WHERE follower_user_id = ?)
                    and u.user_id != ?
                ORDER BY
                    u.follower_count DESC,
                    u.post_count DESC,
                    RAND() -- Randomly sort to increase diversity
                LIMIT ? OFFSET ? `;
```

Specific effects: After the user logs in, the system will recommend to the user on the right the farmers who are worthy of transformation. The effect is shown in Figure 24 (left). After clicking "Follow", "Following" will be displayed. After a successful follow, the user will be added to the follow list and disappear from the recommendation list, as shown in Figure 24 (right).

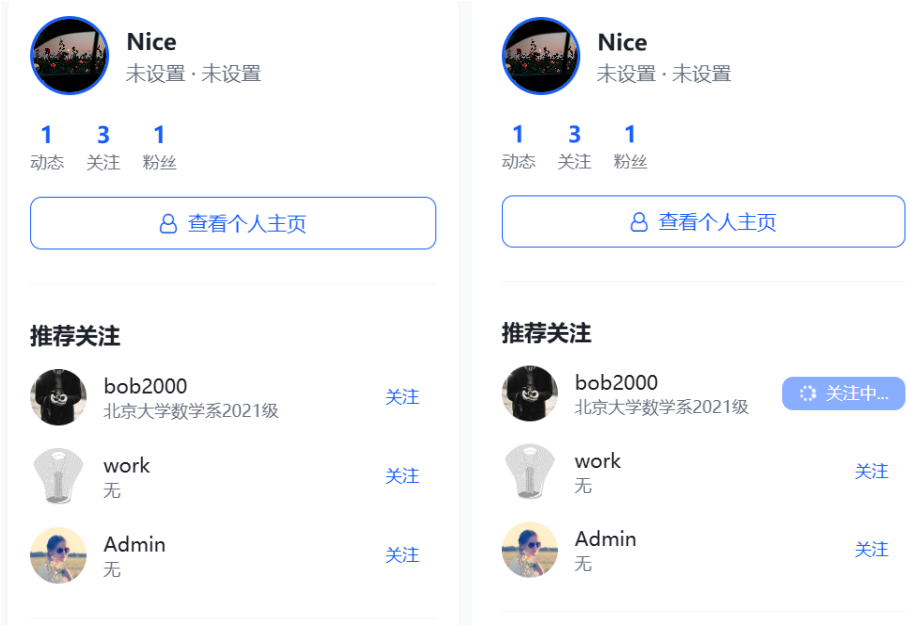


Figure 24. User Recommendation Effect

3.5.2 Clock in and sign in

Overall implementation: To encourage users to use this platform more, a check - in function is added in the lower right corner. By displaying the number of check - in days in the current month, the checked - in days are marked with corresponding colors to represent successful check - in (referring to the check - in function on Leetcode). Also, when checking in, users can add their current mood. When clicking on a specific check - in day, the mood at that time will be displayed above, which is equivalent to a small diary function. This can attract young users to use the platform. Meanwhile, when the check - in is successful, explicit messages like "Great!" will be provided to encourage users to check in.

Specific effects: Click the sign-in button at the bottom right corner to get effect shown in Figure 25.

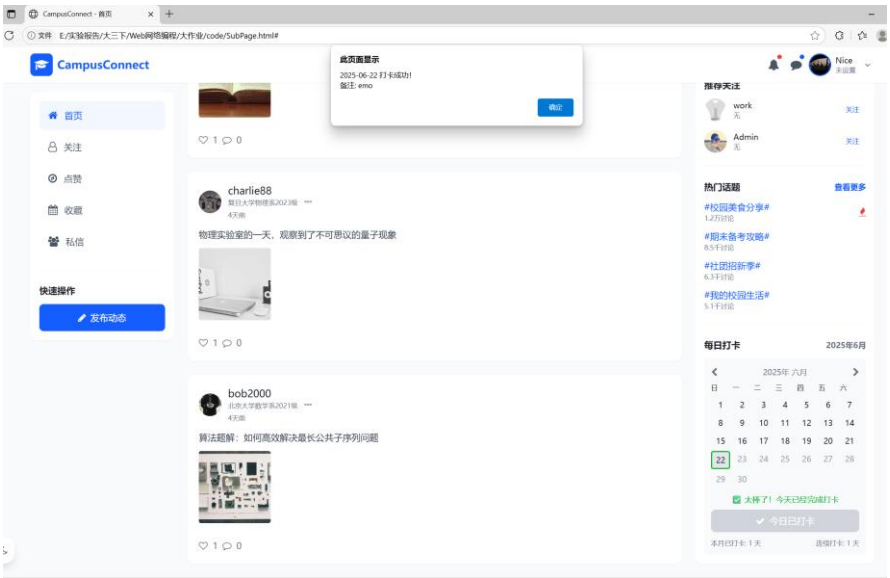


Figure 25. Punching-in effect

3.5.3 Search function

Overall implementation: Add a search function that supports searching for users with similar names and posts whose entry content contains the search terms. The actual functionality is mainly accomplished by SQL statements. Find the corresponding users or posts and return them, and then render the corresponding content in the modal box. The main functional statements are as follows:

```
// Search for users
const userSearchQuery = `
    SELECT user_id, username, avatar_url, school_info, gender
    FROM Users
    WHERE username LIKE ?
    LIMIT 5;
`;
// Search for posts
const postSearchQuery = `
    SELECT
        p.post_id, p.content, p.media_url, p.created_at, p.like_count, p.comment_count,
        u.user_id AS author_user_id,
        u.username AS author_username,
        u.gender AS author_gender,
        u.avatar_url AS author_avatar_url
    FROM Posts p
    JOIN Users u ON p.user_id = u.user_id
    WHERE p.content LIKE ?
    ORDER BY p.created_at DESC
    LIMIT 10;
`;
```

Specific effects: Click on the search box, search for the keyword "Nice", and the result is a user, as shown in Figure 26 (left). When the search keyword is "love", the result is a post, as shown in Figure 26 (right).

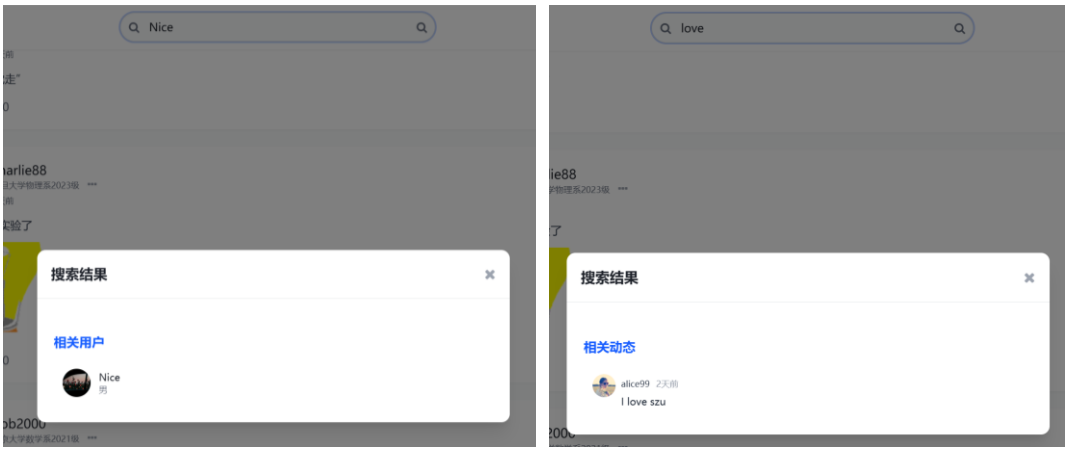


Figure 26. Search Result 1 (left), Search Result 2 (right)

四、Backend Service Design

This backend adopts API development based on the Express framework. It connects to the local MySQL database through the mysql2/promise module, handles file uploads through Multer, encrypts passwords using bcryptjs, handles cross - origin requests through the cors middleware, and manages environment configurations through dotenv. The overall database configuration is as follows:：

```
const express = require('express');                 // 引入 Express 框架
const mysql = require('mysql2/promise');            // 引入 MySQL2 的 promise 版本
const bodyParser = require('body-parser');
```

```
const cors = require('cors');
const fs = require('fs');
const dotenv = require('dotenv');
const bcrypt = require('bcryptjs'); // 用于密码加密
dotenv.config();

const app = express();
const PORT = 3000;
app.use(cors());
app.use(bodyParser.json());

const multer = require('multer');
const path = require('path');

// 确保上传目录存在
const uploadDir = path.join(__dirname, 'public', 'uploads');
if (!fs.existsSync(uploadDir)) {
    fs.mkdirSync(uploadDir, { recursive: true });
}

// 配置 Multer 存储引擎
const storage = multer.diskStorage({
    destination: function (req, file, cb) {
        cb(null, uploadDir);
    },
    filename: function (req, file, cb) {
        const uniqueSuffix = Date.now() + '_' + Math.round(Math.random() * 1e9);
        const ext = path.extname(file.originalname);
        cb(null, uniqueSuffix + ext);
    }
});
// 创建 multer 实例
const upload = multer({
    storage: storage,
    limits: { fileSize: 10 * 1024 * 1024 } // 限制文件大小为 10MB
});
// 静态资源目录配置
app.use('/uploads', express.static(path.join(__dirname, 'public', 'uploads')));
let connection; // 声明连接变量
// 初始化数据库连接
async function initDb() {
    try {
        connection = await mysql.createConnection({
            host: 'localhost',
            user: 'root',
            password: '********',
            database: 'web'
```

```
        });
        console.log('数据库连接成功');
    } catch (err) {
        console.error('数据库连接失败:', err);
        process.exit(1); //  连接失败时退出应用
    }
}
```

In this project, a total of 8 tables were created. The visualization using this tool is as shown in Figure 27:

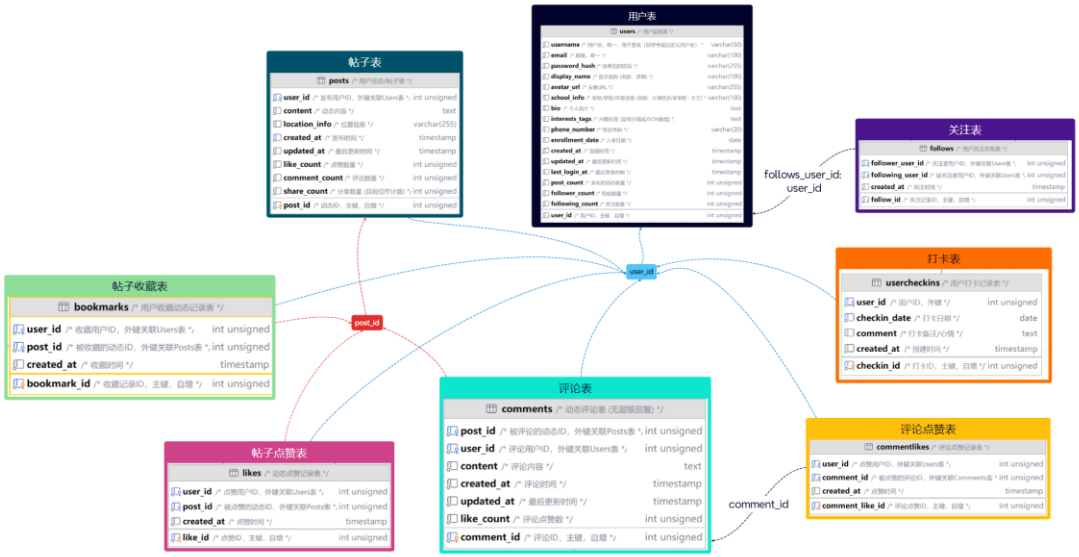


Figure 27. Database Table View

五、Project Summary and Reflection

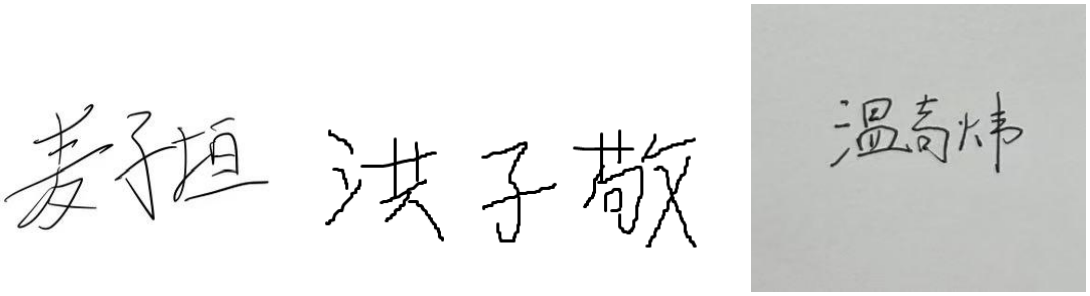
In this experiment, HTML, CSS, and JavaScript were used for web design, starting from scratch and building up step by step. During the project implementation, I also gained a lot of team development experience. For example, the team maintained a document for unified operations to achieve information sharing. At the same time, the team leader needed to unite the team members in the group and urge the completion of each part, which is the foundation for ensuring the complete progress of a project. In addition, I also obtained a lot of front - end development experience, such as how to use developer tools for debugging, and that a 400 return indicates a client - side problem while a 500 return indicates a server - side problem and other practical information. All in all, this experiment has been very rewarding.

诚信承诺：

本人郑重承诺在该课程论文完成的过程中不发生任何不诚信现象，一切不诚信所导致的后果均由本人承担。

Integrity Commitment:

I/We solemnly promise that I/we have not plagiarized during the completion of this course paper, and all the consequences caused by plagiarism will be borne by me/us.



签名拍照附件:

Name and Digital Signature of all students in the group

Date:2025.6.22